Lab 3: Custom Event Handling – Stock Price Alert System

🧠 Goal

Learn how to create and handle custom events in C#. You'll simulate a stock that raises an event when its price changes, and a monitor that responds.

🛠️ Step-by-Step Instructions

✅ Step 1: Create the Project

1. Open Visual Studio or your preferred C# development environment.
2. Create a new Console App project named `EventStockMonitorLab`.

✅ Step 2: Define the EventArgs Class

Create a class for passing event data:

```
public class PriceChangedEventArgs : EventArgs
{
    public decimal OldPrice { get; set; }
    public decimal NewPrice { get; set; }
}
```

✅ Step 3: Create the Stock Class

Define the stock with a `PriceChanged` event:

```
public class Stock
{
    public string Symbol { get; set; }
    private decimal price;

    public event EventHandler<PriceChangedEventArgs> PriceChanged;

    public decimal Price
    {
        get => price;
        set
```

```
        {
            if (price != value)
            {
                var oldPrice = price;
                price = value;
                PriceChanged?.Invoke(this, new PriceChangedEventArgs { OldPrice = oldPrice,
NewPrice = value });
            }
        }
    }
}
```

✅ Step 4: Create the Stock Monitor

Define a class that subscribes to the stock's event:

```
public class StockMonitor
{
    public void Subscribe(Stock stock)
    {
        stock.PriceChanged += HandlePriceChanged;
    }

    private void HandlePriceChanged(object sender, PriceChangedEventArgs e)
    {
        Console.WriteLine($"Stock price changed from {e.OldPrice:C} to {e.NewPrice:C}");
    }
}
```

✅ Step 5: Simulate Price Changes

In `Main()`:

```
var stock = new Stock { Symbol = "ACME", Price = 100.0m };
var monitor = new StockMonitor();
monitor.Subscribe(stock);

stock.Price = 101.0m;
stock.Price = 105.5m;
stock.Price = 105.5m; // Should not raise event
stock.Price = 99.9m;
```

🌟 Stretch Goal: Subscribe with a Lambda

Instead of a class, try subscribing directly with a lambda:

```
stock.PriceChanged += (s, e) => Console.WriteLine($"[Lambda] Price: {e.OldPrice:C} → {e.NewPrice:C}");
```

✅ What You Learned

- How to define and raise custom events
- How to use `EventHandler<T>` and `EventArgs`
- How to subscribe using both methods and lambdas

🧪 Next Lab

In Lab 4, you'll build a mini app that uses delegates, lambdas, and events to simulate a real-world notification system.